SubsFlow — Frontend File Structure & Design Guide

Interactive React Dashboard, Real-Time Telemetry & Gateway Sandbox

Framework: React 19 + Vite 8 **Design System:** Vanilla Glassmorphism CSS **State:** Unified Hook & Callback Telemetry

1. Frontend Architecture & Core Concept

The SubsFlow frontend is built as a single-page operational control center (SPA) with a glassmorphic dark-mode design system. Rather than being a static CRUD dashboard, it acts as an **interactive visual sandbox and developer cockpit** that demonstrates real-time billing orchestration, simulated payment faults, dunning queues, and outbox event streaming.

Core Frontend Design Principles:

- **Dual Authentication Modes:** Supports JWT Token auth and tenant API Key isolation headers (X-API-Key).
- **Live Telemetry & Performance Logger:** Every outgoing API call automatically measures millisecond latency and logs full request/response payloads in real time.
- **Interactive Chaos Testing (Gateway Sandbox):** Allows operators to inject payment failures (insufficient funds, card expired, timeouts) to test dunning retries in real time.
- **Dynamic Pipeline Visualizer:** Animated stage-by-stage node diagram visualizing the full subscription lifecycle.

2. Frontend Directory & File Hierarchy

frontend/
■■■ **package.json** — React 19, Vite 8 build configurations & scripts
■■■ **vite.config.js** — Vite dev server & proxy settings forwarding /api to Spring Boot (port 8080)
■■■ **index.html** — Root HTML template with Google Fonts (Inter, Fira Code)
■■■ **src/**
  ■■■ **main.jsx** — Application bootstrap & ReactDOM rendering
  ■■■ **index.css** — Global design tokens, CSS variables, dark palette & reset
  ■■■ **App.jsx & App.css** — Root shell, topbar, toast context, telemetry state
  ■■■ **api.js** — Centralized HTTP client, JWT storage, latency timer, API endpoints
  ■■■ **components/** — Modular feature panels & UI widgets
    ■■■ **LoginScreen.jsx / .css** — Tenant registration & API Key login interface
    ■■■ **Dashboard.jsx / .css** — KPI summary bar, tab navigation, data coordination
    ■■■ **PipelineVisualizer.jsx / .css** — Animated 6-stage architecture lifecycle visualizer
    ■■■ **PlansPanel.jsx / .css** — Billing plan cards & dynamic tier selector
    ■■■ **SubscriptionsPanel.jsx / .css** — Active subscriptions list & cancellation handler
    ■■■ **ChangePlanForm.jsx** — Mid-cycle plan upgrade/downgrade with proration preview
    ■■■ **UsageForm.jsx** — Metered telemetry event ingestion generator
    ■■■ **InvoicesPanel.jsx / .css** — Invoices ledger with itemized breakdown drawer
    ■■■ **GatewaySandbox.jsx / .css** — Chaos testing sandbox for simulated payment declines & dunning
    ■■■ **ApiLog.jsx / .css** — Real-time HTTP network & telemetry console with JSON viewer
    ■■■ **ToastNotification.jsx / .css** — Non-blocking floating notification feedback system

3. Component Details & Functional Responsibilities

Component	Files (.jsx & .css)	Core Idea, State & Functionality
App Root & Shell	App.jsx App.css	Maintains global tenant session, active telemetry log array (capped at 100 entries), and active toast notifications. Houses the glassmorphic top header and switches between Login and Dashboard views.
Central API Layer	api.js	Provides a unified fetch client that automatically attaches JWT tokens, measures execution latency using performance.now(), standardizes error responses (502, 503, 400), and handles localStorage persistence.
Login & Onboarding	LoginScreen.jsx LoginScreen.css	Dual-action portal allowing new tenants to register (generating a unique API Key) or existing tenants to log in with an existing key. Preloads quick demo credentials for instant evaluation.
Operational Dashboard	Dashboard.jsx Dashboard.css	Calculates dynamic KPIs (Estimated MRR, Active Subscriptions, Total Invoiced) and manages main tabs (**Pipeline**, **Plans & Subs**, **Invoices**, **Sandbox**).
Pipeline Visualizer	PipelineVisualizer.jsx PipelineVisualizer.css	Interactive 6-step diagram: *1. Ingestion → 2. Rating → 3. Invoicing → 4. Payment → 5. Dunning → 6. Outbox*. Users can click any node to view architecture mechanics and state transitions.
Plans & Subscriptions	PlansPanel.jsx SubscriptionsPanel.jsx	Renders available pricing plans with badges (Flat, Per Unit, Tiered) and lists current active subscriptions with status indicators and cancellation triggers.
Plan Change & Usage	ChangePlanForm.jsx UsageForm.jsx	Forms to trigger mid-cycle plan migrations (demonstrating proration and idempotency headers) and emit metered usage events (e.g., 500 API calls) directly into the billing engine.
Invoices Ledger	InvoicesPanel.jsx InvoicesPanel.css	Displays invoice table with status tags (PAID, UNPAID, VOID) and expands into an itemized modal showing base plan charges, usage overages, and payment transaction metadata.
Gateway Sandbox	GatewaySandbox.jsx GatewaySandbox.css	Chaos engineering console to test payment failure handling. Operators can simulate credit card declines, network timeouts, and replay identical idempotency keys to verify zero duplicate charges.
Telemetry & ApiLog	ApiLog.jsx ApiLog.css	Real-time terminal log viewer. Displays HTTP method (GET/POST), status badges (200 OK, 400 Bad Request), latency in milliseconds, and format-formatted JSON inspector.

4. Frontend Data Flow & State Lifecycle

The diagram below outlines how user actions in the frontend flow through the API layer into Spring Boot and return structured feedback to the UI:

1. User Interaction (Action Triggered)

User clicks an action in the UI (e.g. Subscribing to a plan, emitting usage, or simulating a gateway failure).

2. Centralized Request Builder (api.js)

api.js captures the start timestamp, attaches the tenant's JWT Bearer token and X-API-Key, generates a unique Idempotency-Key if required, and executes the HTTP fetch.

3. Spring Boot Backend Execution

Spring Boot's TenantAuthFilter validates the tenant, sets Postgres RLS, and executes the requested operation atomically.

4. Response & Telemetry Hook (addLog Callback)

Upon receiving the HTTP response, api.js computes elapsed milliseconds. The result is dispatched to addLog(), immediately appending a real-time entry to ApiLog.jsx.

5. State Mutation & Toast Feedback

The relevant component refreshes its state (e.g. fetchSubscriptions()), recalculates MRR metrics, and triggers ToastNotification.jsx to show a success or error alert.

5. Styling System & UI Tokens

SubsFlow uses a modern, bespoke CSS design system implemented without heavy third-party UI dependencies for optimal loading speed and zero bloat:

Token Category	CSS Variable / Class	Design Purpose
Glassmorphism	.glass-panel backdrop-filter: blur(16px)	Translucent slate background (rgba(30, 41, 59, 0.75)) with soft border highlights for a sleek dark aesthetic.
Color Accents	--primary-indigo: #4f46e5 --success-emerald: #10b981	Semantic color tokens for status badges (ACTIVE: green, PAST_DUE: amber, CANCELLED: red, VOID: slate).
Typography	Inter, Fira Code	Clean sans-serif typography for readable interfaces paired with monospaced font for API keys, JSON logs, and currency amounts.
Micro-Animations	@keyframes pulse-node @keyframes toast-slide-in	Smooth CSS animations for pipeline node transitions, pulse indicators on healthy nodes, and toast entries.